Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:19-03-2026**	**Enrolment No:92301733024**

Aim: To understand the process of convolution over the image and apply over the classification problem

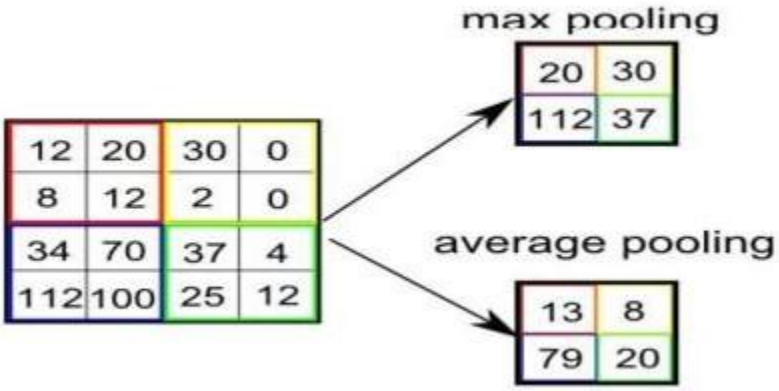
IDE: Google Colab

Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

What is Pooling Layer?

Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.



What is Convolutional Layer?

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:19-03-2026**	**Enrolment No:92301733024**

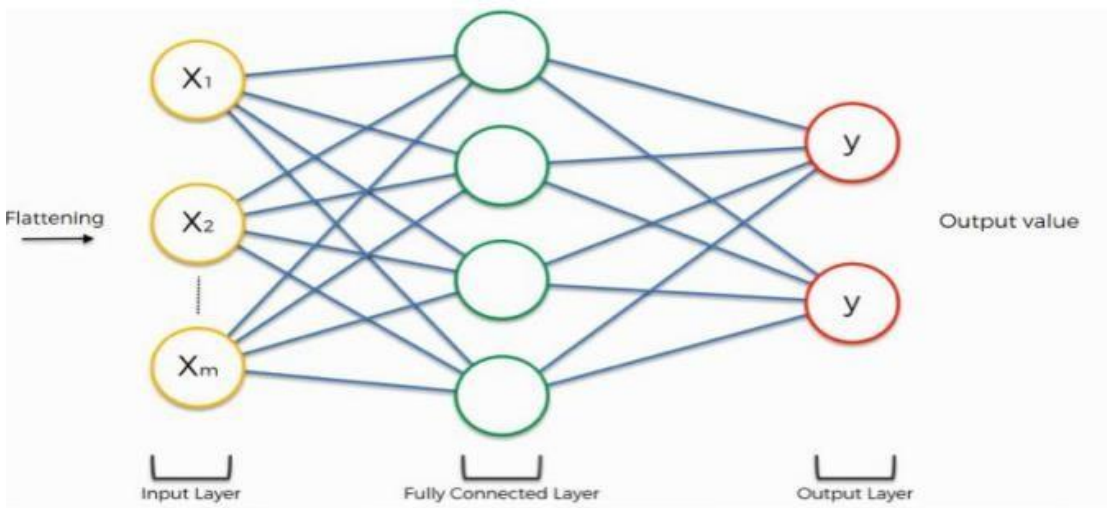
What is Fully Connected Layer?

A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

Input Layer: It takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

Fully Connected Layer: It takes the inputs from the feature analysis and applies weights to predict the correct label.

Output Layer: It gives the final probabilities for each label.



ReLU Layer: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:19-03-2026**	**Enrolment No:92301733024**

7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

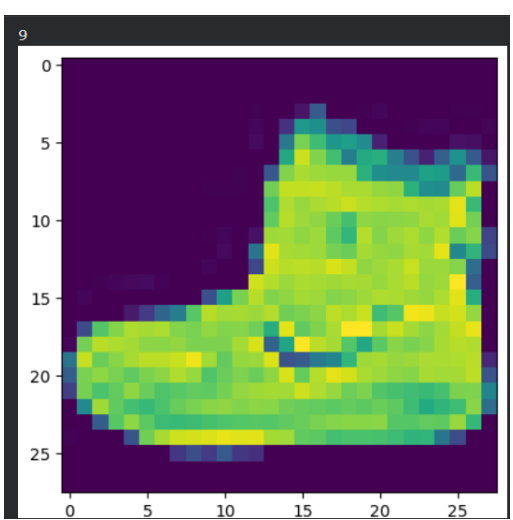
Program (Code):

```
#import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import keras
from keras.datasets import fashion_mnist
import keras.models as models
import keras.layers as layers
from keras import regularizers
from keras.layers import Dropout

import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense,Activation,Flatten,Dropout,Conv2D, MaxPooling2D, LeakyReLU
from tensorflow.keras.utils import to_categorical   #for the categorical data
print(keras.__version__)
```

```
3.13.2
```

```
#import the dataset #x: Images and Y: labels of the images
(train_X,train_Y),(test_X,test_Y)=fashion_mnist.load_data()
plt.imshow(train_X[0])
print(train_Y[0]
```



```
print(train_X.shape,train_Y.shape)
```

```
(60000, 28, 28) (60000,)
```

```
print(test_X.shape, test_Y.shape)
```

```
(10000, 28, 28) (10000,)
```

```
classes=np.unique(train_Y)
nClasses=len(classes)
print(classes)
print(nClasses)
```

```
[0 1 2 3 4 5 6 7 8 9]
10
```

```
train_X=train_X.reshape(-1,28,28,1)
test_X=test_X.reshape(-1,28,28,1)
print(train_X.shape,train_Y.shape)
print(test_X.shape, test_Y.shape)
#Range of pixel values for each type of image
#Black-White: 0-1
#Grayscale: 0-255
#RGB: (0-255)*3
#converting the values of the pixels from 0-1
#type-conversion
train_X=train_X.astype('float32')
test_X=test_X.astype('float32')
train_X=train_X/255
test_X=test_X/255
train_Y_one_hot=to_categorical(train_Y)
test_Y_one_hot=to_categorical(test_Y)
```

train_Y[0]

```
np.uint8(9)
```

train_Y_one_hot[0]

```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 1.])
```

#Model training task

from sklearn.model_selection import train_test_split
train_X,valid_X,train_label,valid_label=train_test_split(train_X,train_Y_one_hot,test_size=0.2,random_state=13)

train_X.shape,valid_X.shape,train_label.shape,valid_label.shape

```
((48000, 28, 28, 1), (12000, 28, 28, 1), (48000, 10), (12000, 10))
```

#Basic CNN model Develop

#Regularization and Dropout
#2nd CNN MODEL DEVELOPMENT

batch_size=64

epochs=20
num_classes=10

model=tf.keras.Sequential()

#1st hidden layer

model.add(tf.keras.layers.Conv2D(32,(3,3),activation="linear",padding='same'))

model.add(tf.keras.layers.LeakyReLU(alpha=0.1))

model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))

#2nd hidden layer

model.add(tf.keras.layers.Conv2D(64,(3,3),activation="linear",padding='same'))

model.add(tf.keras.layers.LeakyReLU(alpha=0.1))

model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))

#3rd hidden layer

model.add(tf.keras.layers.Conv2D(128,(3,3),activation="linear",padding='same'))

model.add(tf.keras.layers.LeakyReLU(alpha=0.1))

model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))

model.add(tf.keras.layers.Flatten())

```
model.add(tf.keras.layers.Dense(128,activation='linear'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.Dense(num_classes,activation='softmax'))#output final layer
model.compile(loss=tf.keras.losses.categorical_crossentropy,optimizer=tf.keras.optimizers.Adam(),metrics=['accuracy'])
48000/64
```

```
750.0
```

```
model_train=model.fit(train_X,train_label,batch_size=64,epochs=20,verbose=1,validation_data=(valid_X,valid_label))
testing_evaluation=model.evaluate(test_X,test_Y_one_hot)
testing_evaluation  #loss,accuracy
```

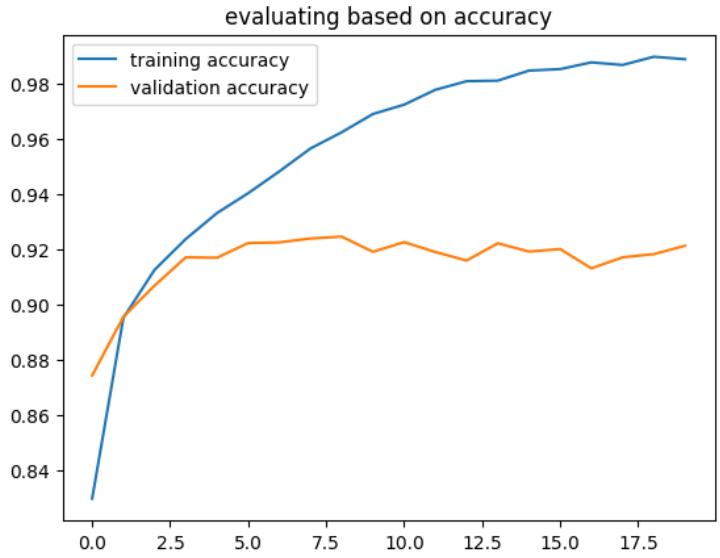
```
[0.4289424419403076, 0.9211999773979187]
```

```
model_train.history
```

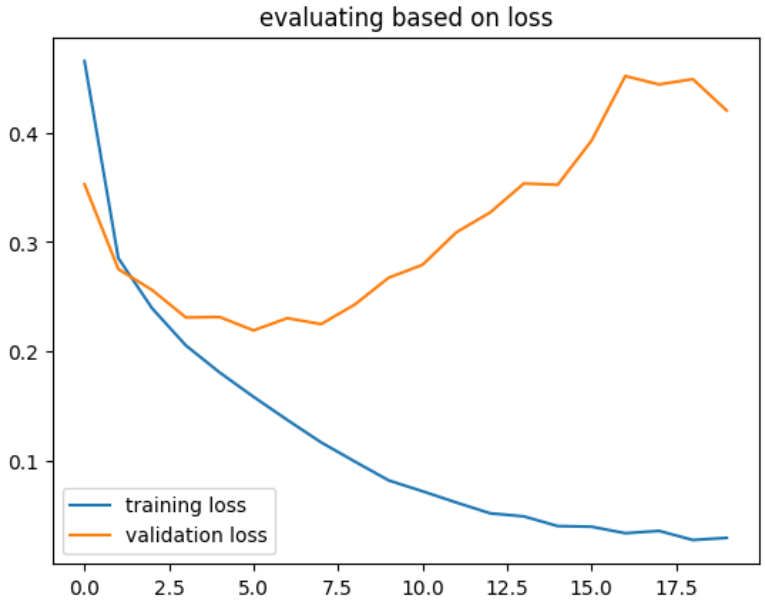
```
{'accuracy': [0.8296874761581421,
  0.8952083587646484,
  0.9126250147819519,
  0.9237916469573975,
  0.9332083463668823,
  0.940416693687439,
  0.9483333230018616,
  0.9566458463668823,
  0.9625208377838135,
  0.9691458344459534,
  0.9725208282470703,
  0.9779166579246521,
  0.9810208082199097,
  0.9812291860580444,
  0.9848750233650208,
  0.9854166507720947,
  0.9878541827201843,
  0.9869166612625122,
  0.9898958206176758,
  0.9890000224113464],
 'loss': [0.4653213620185852,
  0.2852170169353485,
  0.23933947086334229,
```

```
accuracy=model_train.history['accuracy']
val_accuracy=model_train.history['val_accuracy']
loss=model_train.history['loss']
val_loss=model_train.history['val_loss']
epochs=range(len(accuracy))
plt.plot(epochs,accuracy,label='training accuracy')
plt.plot(epochs,val_accuracy,label='validation accuracy')
```

```
plt.title('evaluating based on accuracy')
plt.legend()
plt.show()
```




```
plt.plot(epochs,loss,label='training loss')
plt.plot(epochs,val_loss,label='validation loss')
plt.title('evaluating based on loss')
plt.legend()
plt.show()
```



Regularization

```
regmodel=tf.keras.Sequential()
#1st hidden layer
regmodel.add(tf.keras.layers.Conv2D(32,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.25))
#2nd hidden layer
regmodel.add(tf.keras.layers.Conv2D(64,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.25))
#3rd hidden layer
regmodel.add(tf.keras.layers.Conv2D(128,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.40))

regmodel.add(tf.keras.layers.Flatten())
regmodel.add(tf.keras.layers.Dense(128,activation='linear'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.Dropout(0.3))
regmodel.add(tf.keras.layers.Dense(num_classes,activation='softmax'))#output final layer
regmodel.compile(loss=tf.keras.losses.categorical_crossentropy,optimizer=tf.keras.optimizers.Adam(),metrics=['accuracy'])
reg_model_train=regmodel.fit(train_X,train_label,batch_size=64,epochs=20,verbose=1,validation_data=(valid_X,valid_label))
```

Pre Lab Exercise:

a. **What is Convolution process? Explain giving example.**
   Convolution is the process of applying a filter on an image.It helps to extract features like edges and patterns.
   Example: Detecting edges in an image using a filter.

b. **What are different layers in CNN model?**
   - Convolutional Layer
   - Pooling Layer
   - Fully Connected Layer

c. **What is the requirement of the pooling layer in the CNN model?**

   Pooling layer reduces the size of the feature map. It decreases computation and memory usage. It also helps to remove noise from the data.

d. **What is the requirement of the use of ReLU activation function after convolution step?**
   ReLU removes negative values from the output. It introduces non-linearity in the model. It helps in faster training of the network.

Observation and Result Analysis:

a. **Nature of the dataset**
   The dataset contains image data of size 28×28. It is a grayscale dataset. Each image belongs to a specific class.

b. **Training Process without regularization**
   The model is trained using CNN layers. Training accuracy increases over epochs. Validation accuracy may fluctuate due to overfitting.

c. **After regularization in the training Process**
   Dropout is applied to reduce overfitting. Validation accuracy improves. The model becomes more generalized.

d. **Observation over the Learning Curves**

   Accuracy increases with epochs. Loss decreases gradually. Regularization gives smoother curves and better performance.

Post Lab Exercise:

a. **Why CNN is preferred over ANN for images**

   CNN captures spatial features of images. It uses filters to detect patterns. ANN cannot efficiently handle image data.

b. Can CNN be applied over Text data? If yes, then how. If no, then why?

Yes, CNN can be used for text data. Text is converted into numerical vectors (embeddings). Then convolution is applied to extract features.

c. What is the role of dropout layer?

Dropout randomly removes some neurons during training. It helps to reduce overfitting. It improves model generalization.

d. What will happen if maxpooling is replaced with minpooling?

Min pooling selects the smallest values. Important features may be lost. It reduces model performance compared to max pooling.